Quantum Neural Networks for Image Classification

Campbell Brown, Irene Kihara, Opal Rosebrock,
Group name: Quantum Chargers
University of Alabama in Huntsville
Mentor: Dr. Dinh Nguyen
Quantum Hackathon 2026 QML Submission

Abstract

Quantum machine learning have emerged as a promising approach to machine learning by integrating quantum computing principles with classical neural network architectures. This work investigates the application of QML for image classification, focusing on their learning capabilities, convergence behavior, and classification performance over multiple training epochs. The proposed approach leverages quantum computing principles, including superposition, entanglement, and parameterized quantum circuits, to process image features and perform classification tasks. The performance of the QML is evaluated by analyzing training loss and classification accuracy across successive epochs. Simulation results demonstrate that the QML model exhibits convergence during training, as reflected by a reduction in loss and an improvement in classification accuracy. Furthermore, the learning behavior of the model is examined through graphical representations of accuracy and loss to assess its training stability and effectiveness. The proposed approach is evaluated using two benchmark datasets, MNIST and CIFAR-10, which consist of grayscale handwritten digits and color images of ten distinct object categories, respectively.

I. Introduction

Quantum Machine Learning (QML) has emerged as a promising research field that integrates the principles of quantum computing with classical machine learning to address complex computational and learning tasks. By leveraging fundamental quantum mechanical properties such as superposition, entanglement, and quantum parallelism, QML explores new approaches to data processing, optimization, and pattern recognition. Within this field, Quantum Neural Networks (QNNs) have gained significant attention as learning models that utilize parameterized quantum circuits, quantum gates, and entanglement operations to process information and learn meaningful data representations. In hybrid quantum-classical architectures, quantum circuits are employed for feature processing and prediction, while classical optimization algorithms update trainable circuit parameters to minimize a predefined loss function. QNNs offer a promising framework for investigating quantum-based learning approaches in applications such as image classification, where classical image features can be encoded into quantum states and processed through trainable quantum circuits. Despite their potential, QNNs face several practical challenges, including limited qubit availability, measurement noise, hardware imperfections, and computational complexity, which can affect classification accuracy, training stability, and model convergence. In particular, the probabilistic nature of quantum measurements introduces statistical uncertainty into circuit outputs, potentially influencing prediction reliability and optimization performance. Increasing the number of measurement shots can reduce statistical fluctuations and improve the precision of estimated outputs, although this improvement comes at the cost of increased circuit executions and computational resources. This project presents an effort to design, implement, and evaluate QNNs for image classification using real-world datasets. The proposed approach integrates classical image preprocessing, quantum feature encoding, parameterized quantum circuits, and classical optimization techniques to investigate the feasibility and effectiveness of quantum-based image recognition. Furthermore, the study examines the impact of measurement noise and different numbers of measurement shots on classification accuracy, loss behavior, training stability, and convergence. The overall objective is to assess the practical applicability of QNNs for real-world image classification.

II. Proposed QNN Model

The Quantum Neural Network (QNN) implemented in this study consists of a four-qubit quantum circuit containing two trainable layers. The model utilizes quantum rotation gates, including RX, RY, and RZ, to manipulate the quantum states of individual qubits. Each layer also incorporates Controlled-NOT (CNOT) gates to introduce entanglement between neighboring qubits, allowing the network to capture relationships within the input data. Each trainable layer contains 12 adjustable rotation parameters, resulting in a total of 24 trainable parameters across the network.

Before entering the quantum circuit, the MNIST and CIFAR-10 images are resized to 2×2 pixels to reduce computational complexity. CIFAR-10 images are additionally converted from RGB to grayscale by averaging their color channels. The resulting four pixel values are encoded into the four qubits using RX rotation gates. Following the two trainable layers, Pauli-Z measurements are performed on the qubits, with the first two measurement outputs used for binary classification.

The QNN was evaluated in 1 shot. A shot represents a single measurement of a quantum circuit, with multiple shots allowing the measurement expectation values to be estimated more accurately. Increasing the number of shots generally reduces measurement variability, although it also increases the measurement cost. The model uses simulated finite-shot measurements rather than physical quantum hardware.

A. Encoding

The first stage of utilizing the QNN, known as encoding, is to encode the classical data of an image or other set of data into a quantum state that a quantum device can utilize. Put simply, the original data is modified to be read in quantum language. Of the two popular ways to encode data, we used angle encoding for our QNN. Angle encoding is when each piece of classical data is converted to the rotation angle of a quantum bit.

An n x n image is transformed into a vector with n to the power of 2 pixel intensities. These are divided into small squares, each assigned to one or more qubits. Each qubit represents a specific part of the overall image.

$$|\psi(\mathbf{x})\rangle = \bigotimes_{i=0}^{n-1} R_a(x_i)|0\rangle \tag{1}$$

B. Parameterized Quantum Circuit

In the second stage, the QNN uses a parameterized quantum circuit to process the encoded data. This is a set of quantum operations that act as filters and simplifiers. Similar to how a classic CNN identifies patterns and reduces data size. The parameterized quantum circuit is made up of layers, each integrating two types of operations as follows.

The first operation is convolution which uses quantum filters on small groups of adjacent qubits to find local patterns. These filters are quantum processes with configurable parameters for the layer. Using the same parameters in all neighborhoods ensures consistency and minimizes the number of parameters.

The second operation is pooling which decreases the number of qubits. It employs a quantum procedure known as isometry to shrink the system while maintaining its quantum nature.

C. Quantum Measurement

In the third stage, after processing the data through the parameterized quantum circuit, we obtain the final quantum state. To make a prediction (categorizing the image as dog, boat etc.), we measure a quantum characteristic known as observable(O). The expected outcome is explained as y = g(¡O¿pL) = g(Tr[Opl]) where O is a quantum observable chosen for the task at hand. For example, it could follow symmetries like parity for specific issues; ¡O¿pl is the average value of O when measured in pl; and g() is a simple function that translates the measurement into a usable output, like a class label.

This measurement provides us with the final answer. Fig. 1 shows the structure of a fully quantum QNN model.

QNNs combine the advantages of quantum computing with the pattern finding abilities of classical CNNs. They are very useful for quantum-related tasks such as phase classification in quantum systems, as well as challenges that conventional approaches struggle with.

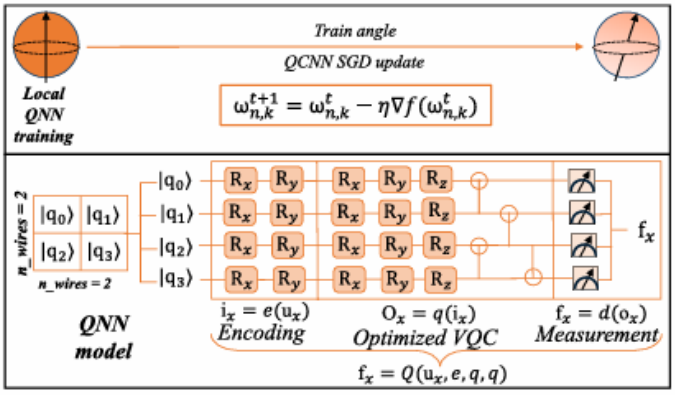


Fig. 1. Example of a fully quantum QCNN block and training loop.

III. Simulation Results

A. Dataset Explanation

The selected datasets were MNIST and CIFAR-10. MNIST consists of black and white graphics containing numbers of 0-9 whereas CIFAR contains various images of different RGB values depicting a dog, bird, ship, and other categories. The primary goal of this exercise was to use the following databases to evaluate how the differences in complexity and dimensionality affects the overall performance of the QNN. To illustrate the difference the training loss and classification accuracy of the two sets are plotted for each set to allow for a direct comparison of the learning behavior and the effectiveness on the specific sets.

CIFAR-10 contains 50000 training sets and 10000 tests data sets, whereas MNIST contains 60000 training sets and 10000 test sets.

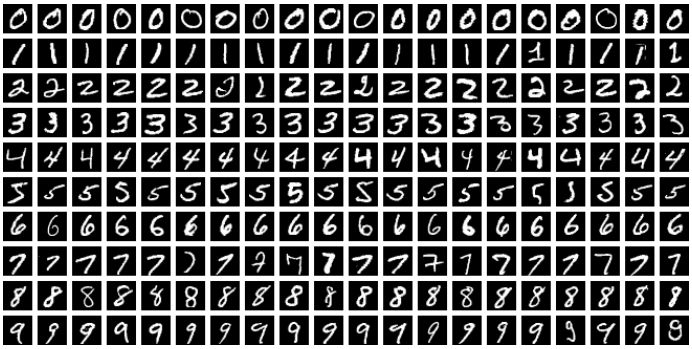


Fig. 2. MNIST Database Set Example



Fig. 3. CIFAR Database Set Example

B. QNN Training Results

The final results illustrate that the less variation the quicker and more accurate the convergence. The primary difference between the two sets is the amount of variance. Where the MNIST data contains only white and black, the CIFAR-10 data contains far more information due to increased image variance due to the difference in colors being added. The final results reflected the accuracy of the MNIST data converging around 89% and the CIFAR-10 dataset converging around 70%. The loss also converged lower to under .54 for the MNIST dataset and under .62 for the CIFAR-10 dataset. Overall the model was more adept at the MNIST dataset however future improvements could be utilized to potentially bridge the gap.

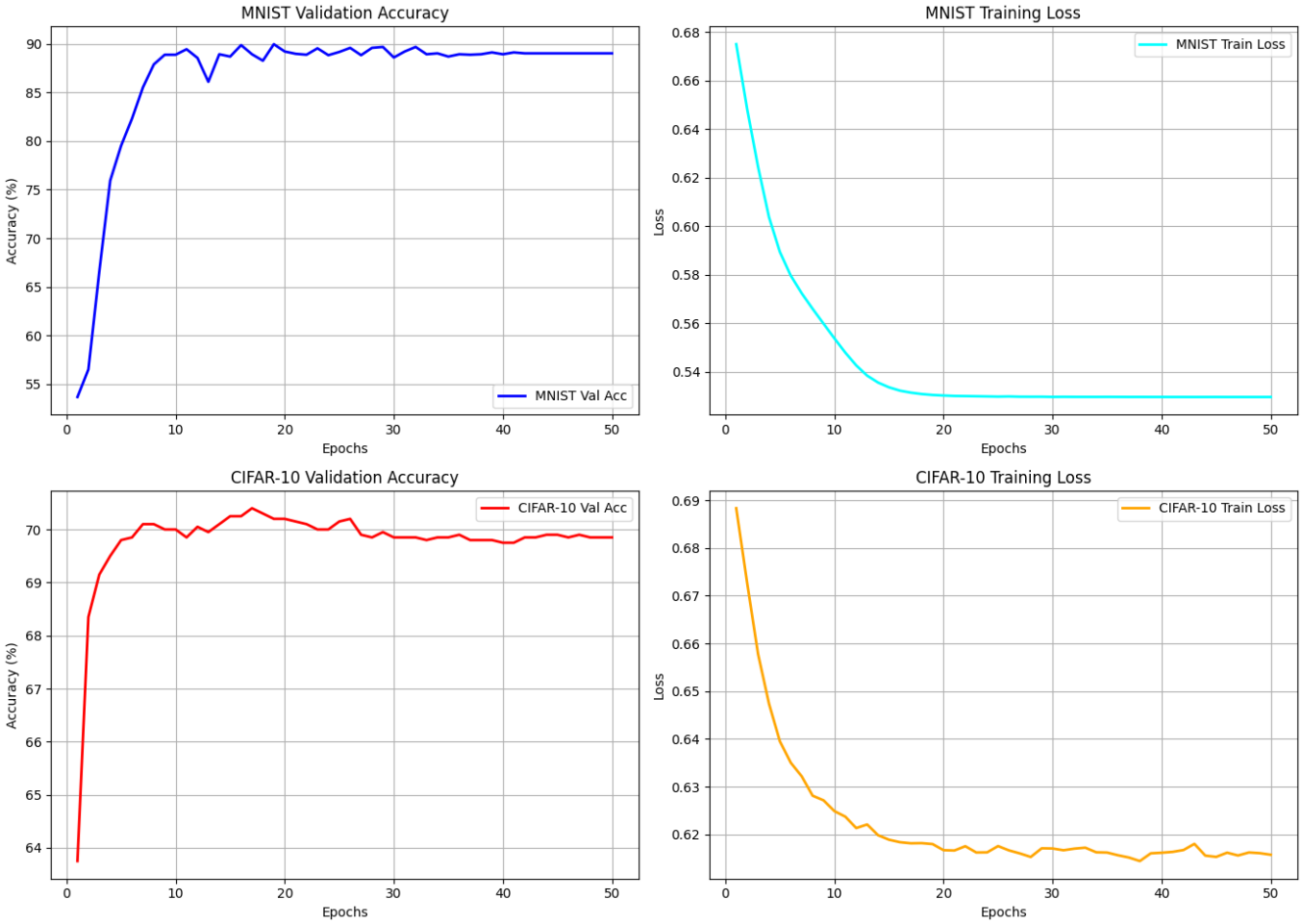


Fig. 4. Final Solutions

IV. Conclusion

This project investigated the use of QNN for image classification for various datasets. The simulation results demonstrate that QNN can perform well and converge stably after a certain number of training epochs. Further improvements such as increased qubits, increased shots, increased epochs, could result in a more stable output though at the expense of increased energy and time.

Github Repo: https://github.com/CampS0up/Quantum_Hackathon.git

V. References

Git Repos:
1. Quantum-Neural-Networks: https://github.com/dinhgit/Quantum-Neural-Networks
2. torchquantum: https://github.com/mit-han-lab/torchquantum
3. MNIST Dataset: https://www.tensorflow.org/datasets/catalog/mnist
4. CIFAR-10 Dataset: https://www.tensorflow.org/datasets/catalog/cifar10